BCT – LP-III Viva Preparation

100 Mixed-Difficulty Questions with Detailed, Simple Answers

How to use this bank: Questions marked with ★ are high-priority. Prepare those first, then revise the remaining questions. Answers are written in simple viva-friendly language.

Coverage: Blockchain, Ethereum, MetaMask, wallets and security, ETH/Wei/Gwei, networks and testnets, transactions and gas, smart contracts, EVM, Solidity, Remix IDE, deployment, Bank Account contract and Student Data contract.

1. Blockchain and Ethereum Fundamentals

★ What is blockchain technology?

Answer: Blockchain is a distributed digital ledger that stores records in blocks linked using cryptography. Copies of the ledger are maintained by multiple computers in a network. Once records are confirmed, changing old data is extremely difficult.

★ How does a blockchain work?

Answer: Transactions are broadcast to the network and checked by participating nodes. Valid transactions are grouped into a block, which is added to the chain through the network's consensus mechanism. Nodes then update their copies of the ledger.

What is a distributed ledger?

Answer: A distributed ledger is a database shared across multiple computers rather than stored at only one central location. Participants maintain and verify copies of the records. Blockchain is one type of distributed ledger.

What is a block in a blockchain?

Answer: A block is a collection of verified transactions and related information. It normally contains data such as a timestamp, transaction references and the hash of the previous block. Linking hashes helps protect the chain from tampering.

What is a hash function?

Answer: A hash function converts input data of any size into a fixed-length output. The same input produces the same hash, while a small input change usually produces a very different output. Blockchains use hashes to identify data and link blocks.

What is decentralization?

Answer: Decentralization means that control and record-keeping are shared among many network participants instead of one central authority. It reduces dependence on a single organization, although the level of decentralization varies between networks.

★ What is Ethereum?

Answer: Ethereum is a blockchain platform that supports programmable smart contracts and decentralized applications. Its native cryptocurrency is Ether (ETH). Developers use it to build applications whose logic can run on the Ethereum network.

★ What is Ether (ETH)?

Answer: Ether is the native cryptocurrency of the Ethereum network. It is used to pay transaction and computation fees, and it can also be transferred between accounts. ETH is the asset; Ethereum is the network.

2. Ethereum Units, Accounts and Wallets

★ What is Wei?

Answer: Wei is the smallest denomination of Ether. One ETH equals 10^18 Wei. Ethereum uses Wei for precise internal calculations because transaction values and balances need very small units.

★ What is Gwei?

Answer: Gwei means giga-Wei and is commonly used to express Ethereum gas prices. One Gwei equals 10^9 Wei, or 0.000000001 ETH. It is easier to read gas prices in Gwei than in very large Wei values.

★ Convert 1 ETH into Wei and Gwei.

Answer: 1 ETH = 1,000,000,000,000,000,000 Wei (10^18 Wei). Since 1 Gwei = 10^9 Wei, 1 ETH = 1,000,000,000 Gwei (10^9 Gwei).

What is the difference between ETH, Wei and Gwei?

Answer: They are denominations of the same currency, Ether. ETH is the main unit, Wei is the smallest unit, and Gwei is commonly used for gas prices. They are not separate cryptocurrencies.

★ What is a cryptocurrency wallet?

Answer: A cryptocurrency wallet is software or hardware that manages the keys used to access blockchain accounts. It lets users view balances, sign transactions and interact with applications. The assets themselves remain recorded on the blockchain.

★ What is MetaMask?

Answer: MetaMask is a self-custody crypto wallet available as a browser extension and mobile app. It lets users manage Ethereum-compatible accounts, connect to decentralized applications, view assets and approve transactions.

Is MetaMask a cryptocurrency or an exchange?

Answer: No. MetaMask is a wallet interface, not a cryptocurrency and not itself a blockchain. It helps users manage keys and interact with networks; buying or swapping assets may be offered through integrated third-party services.

★ What is a public address?

Answer: A public address is an identifier that can be shared to receive cryptocurrency or interact with a blockchain account. On Ethereum it is commonly shown as a hexadecimal string beginning with 0x. Sharing it does not reveal the private key.

★ What is a private key?

Answer: A private key is a secret cryptographic value used to authorize transactions and prove control of an account. Anyone who obtains it can control the associated assets. It must never be shared or entered into untrusted websites.

★ What is a Secret Recovery Phrase (seed phrase)?

Answer: It is a sequence of words used to recover a wallet and derive its accounts. Anyone with the phrase may be able to control the wallet, so it must be stored securely offline and never shared with support staff or websites.

What is the difference between a public key and a private key?

Answer: The private key is secret and is used to create digital signatures. The public key is mathematically related to it and can be used to verify signatures. An Ethereum address is derived from the public key.

What is a custodial wallet versus a self-custody wallet?

Answer: In a custodial wallet, a service provider controls the private keys on the user's behalf. In a self-custody wallet, the user controls the keys or recovery phrase. MetaMask is generally used as a self-custody wallet.

★ What happens when you create an account in MetaMask?

Answer: MetaMask creates or imports wallet key material and displays an account address. The wallet uses the private key to sign approved actions. The account is then usable on supported networks, subject to having the network's required assets for fees.

Can the same MetaMask account be used on different networks?

Answer: Yes. The same account address can generally be used on Ethereum and compatible EVM networks. However, balances, tokens and contract deployments are network-specific, so the selected network must be checked before sending or deploying.

3. Networks, Transactions and Gas

★ What is a blockchain network in MetaMask?

Answer: A network is a particular blockchain environment to which MetaMask connects, such as Ethereum Mainnet or a test network. Each network has its own chain state, transaction history and often its own native fee asset.

★ What is Ethereum Mainnet?

Answer: Ethereum Mainnet is the live public Ethereum network where transactions and smart contract deployments use real ETH and have real economic consequences. It should not be confused with a test network.

★ What is a testnet? Give an example.

Answer: A testnet is a blockchain network used to test transactions and smart contracts without using real-value mainnet funds. Sepolia is a commonly used Ethereum testnet. Testnet ETH is intended for testing and has no official real-world value.

What is the difference between Mainnet and Sepolia?

Answer: Mainnet is the production Ethereum network and uses real ETH. Sepolia is a test network intended for development and testing, where test ETH is used. A contract deployed on Sepolia is not automatically deployed on Mainnet.

What is a faucet?

Answer: A faucet is a service that distributes small amounts of testnet cryptocurrency to developers. It helps them pay testnet gas fees while testing applications. Faucet funds are not real ETH.

★ What is a blockchain transaction?

Answer: A transaction is a cryptographically signed instruction submitted by an account to the network. It may transfer ETH, deploy a contract or call a smart contract function. State-changing transactions require gas.

What are the main fields in an Ethereum transaction?

Answer: Common fields include the sender, recipient, value, nonce, gas limit, fee information and optional input data. A contract call uses input data to specify which function and arguments are being requested.

What is a transaction hash (transaction ID)?

Answer: A transaction hash is a unique identifier calculated for a submitted transaction. It can be used in a block explorer to inspect its status, sender, recipient, value, fee and block information.

★ What is gas in Ethereum?

Answer: Gas measures the computational work required to execute an operation on Ethereum. Transactions pay a fee based on the gas consumed and the price per gas unit. Gas helps prevent network resources from being used without cost.

★ How is an Ethereum gas fee calculated?

Answer: A simplified calculation is gas used multiplied by the effective price per gas unit. For example, 21,000 gas at 10 Gwei costs 210,000 Gwei, which is 0.00021 ETH. Actual fee mechanics can include base fee and priority fee.

What is gas limit?

Answer: Gas limit is the maximum amount of gas that a transaction is allowed to consume. A complex smart contract call generally needs more gas than a simple ETH transfer. Setting it too low can cause execution to run out of gas.

What is gas price?

Answer: Gas price is the amount paid per unit of gas. It is commonly expressed in Gwei. On modern Ethereum transactions, fee settings include a base fee and a priority fee rather than only one legacy gas-price value.

What is base fee?

Answer: The base fee is the minimum per-gas fee determined by Ethereum's protocol for a block. It is burned when the transaction is included. It changes according to how full recent blocks have been.

4. Smart Contracts and Ethereum Architecture

★ What is a smart contract?

Answer: A smart contract is a program deployed on a blockchain that executes according to its coded rules. Users interact with it through transactions or read calls. Once deployed, its code and state are maintained by the blockchain.

What are the advantages of smart contracts?

Answer: They can automate rules, reduce dependence on intermediaries and provide transparent execution. Their outcomes can be independently verified on-chain. However, bugs can be costly and deployed code may be difficult to change.

What are the limitations of smart contracts?

Answer: Smart contracts cannot automatically know real-world information without external data sources. They also have execution costs, limited privacy and risks from programming errors. Incorrectly deployed code can be difficult to repair.

★ What is the Ethereum Virtual Machine (EVM)?

Answer: The EVM is the execution environment that runs Ethereum smart contract bytecode. It defines how contract instructions are processed and how state changes are calculated. EVM-compatible networks implement a similar execution environment.

What is an externally owned account (EOA)?

Answer: An EOA is an Ethereum account controlled by a private key, usually through a wallet such as MetaMask. It can initiate transactions and pay gas. It is different from a contract account, which is controlled by code.

What is a contract account?

Answer: A contract account is created when a smart contract is deployed. It has an address, code and persistent storage. It does not independently initiate transactions; it executes when called through a transaction or another contract.

What is the difference between an EOA and a smart contract account?

Answer: An EOA is controlled by a private key and can initiate transactions. A contract account is controlled by its deployed code and responds to calls. Both have Ethereum addresses and can hold ETH.

What is bytecode?

Answer: Bytecode is the low-level machine code produced by compiling a smart contract. The EVM executes this bytecode, not the original human-readable Solidity source code.

★ What is ABI?

Answer: ABI stands for Application Binary Interface. It describes a smart contract's externally accessible functions, events and data types in a machine-readable format. Tools use the ABI to encode calls and decode returned data.

What is contract deployment?

Answer: Deployment is the process of sending a transaction that creates a smart contract on a blockchain. The compiled bytecode is included in the deployment transaction. After successful execution, the network assigns the contract an address.

Can a smart contract be changed after deployment?

Answer: The code at a deployed contract address is generally immutable. Developers can design upgradeable systems using proxy patterns, but these add complexity and trust considerations. A simple deployed contract cannot just be edited like a local file.

What is a decentralized application (DApp)?

Answer: A DApp is an application that uses smart contracts or blockchain infrastructure for some of its functionality. Its user interface may be a normal website, while blockchain interactions are handled through a wallet and network.

5. Solidity Language and Core Syntax

★ What is Solidity?

Answer: Solidity is a high-level, statically typed programming language used to write smart contracts, especially for Ethereum and EVM-compatible networks. Its syntax is influenced by languages such as C++, Python and JavaScript.

What is the purpose of pragma solidity?

Answer: The pragma statement specifies the Solidity compiler version range that the source file is intended to work with. For example, pragma solidity ^0.8.20; allows compatible compiler versions from 0.8.20 up to, but not including, 0.9.0.

★ What is the SPDX license identifier?

Answer: The SPDX comment declares the source code's license identifier, such as MIT. It helps tools and users understand the licensing terms. It is commonly placed at the top of a Solidity file.

What is a contract in Solidity?

Answer: A contract is the main code unit in Solidity, similar in broad purpose to a class in object-oriented programming. It contains state variables, functions, events, modifiers, structs and other declarations.

★ What are state variables?

Answer: State variables are variables declared inside a contract but outside its functions. Their values are stored persistently on the blockchain as part of the contract's state. Updating them through a transaction generally costs gas.

What are local variables?

Answer: Local variables are declared inside a function or code block. They exist only while that execution is running and are not permanently stored in contract storage unless their values are copied into state variables.

★ Explain uint, uint256 and string in Solidity.

Answer: uint is an unsigned integer type and is equivalent to uint256 by default. uint256 stores non-negative integer values using 256 bits. string stores text, such as a student's name or course.

What is the difference between uint and int?

Answer: uint stores only non-negative integers, while int can store both negative and positive integers. Solidity provides different bit-width variants, such as uint8, uint256, int8 and int256.

What is bool in Solidity?

Answer: bool is a Boolean data type that stores either true or false. It is useful for flags such as whether an account is active, whether a student record exists or whether a condition has been met.

★ What are public, private and internal visibility specifiers?

Answer: public allows a function or variable to be accessed externally, subject to Solidity's rules. private restricts access to the defining contract, while internal allows access within the contract and derived contracts. Visibility does not make blockchain data confidential.

What is a constructor?

Answer: A constructor is a special function that runs once when a contract is deployed. It is commonly used to initialize state variables or set the contract owner. It does not have a regular function name.

★ What is a function in Solidity?

Answer: A function is a block of code that performs a specific operation in a contract. It may read or modify state, accept parameters and return values. External transactions are commonly used to invoke state-changing functions.

What is the difference between view and pure functions?

Answer: A view function can read contract state but cannot modify it. A pure function cannot read or modify contract state and works only with its inputs and local calculations. These declarations help communicate the function's behaviour.

★ What is the difference between memory, storage and calldata?

Answer: storage refers to persistent contract storage. memory is temporary data used during function execution. calldata is read-only temporary input data for external function calls and is often efficient for parameters such as arrays and strings.

6. Payability, Calls and Contract Behaviour

★ What does payable mean in Solidity?

Answer: payable marks a function or address as able to receive Ether. A function that should accept ETH must generally be declared payable. Without payable, sending ETH directly with that call is rejected.

★ What is msg.sender?

Answer: msg.sender is a global variable that contains the address that directly called the current function. It is often used to identify the caller, enforce ownership or record who made a deposit.

★ What is msg.value?

Answer: msg.value gives the amount of Ether, in Wei, sent along with the current message or transaction. It is useful inside payable functions to determine how much ETH was received.

What is the difference between msg.sender and tx.origin?

Answer: msg.sender is the immediate caller of the current function, which may be a user or another contract. tx.origin is the original externally owned account that initiated the transaction. Using tx.origin for authorization can create security vulnerabilities.

★ What is receive() in Solidity?

Answer: receive() is a special external payable function that is called when a contract receives plain Ether with empty calldata. A contract can define only one receive function, and it must be payable.

★ What is fallback() in Solidity?

Answer: fallback() is a special function that runs when no function matches the call data, or in certain cases when Ether is sent without a receive function. It may be payable if it needs to accept Ether.

What is the difference between receive() and fallback()?

Answer: receive() handles plain Ether transfers with empty calldata. fallback() handles calls with unmatched or absent function selectors, and can also receive Ether when no receive function is available. Both must be payable to accept ETH.

What is a modifier in Solidity?

Answer: A modifier is reusable code that adds checks or behaviour before and/or after a function runs. It is commonly used for access control, such as allowing only the owner to call a particular function.

What is a revert in Solidity?

Answer: revert stops the current execution and undoes state changes made during that transaction. It can return an error message or custom error. The transaction may still consume gas for the work performed.

7. Remix IDE and Practical Deployment

★ What is Remix IDE?

Answer: Remix is a browser-based development environment for writing, compiling, deploying and testing Ethereum smart contracts. It includes a code editor, Solidity compiler, deployment panel and debugging tools. It is commonly used for beginner Solidity practicals.

★ What are the steps to deploy a contract using Remix?

Answer: Create or open a Solidity file, write the contract and compile it using the Solidity Compiler. Open Deploy & Run Transactions, select an environment and account, then click Deploy and approve the transaction if a wallet is connected.

What is the Solidity Compiler in Remix?

Answer: The Solidity Compiler converts Solidity source code into EVM bytecode and generates information such as the ABI. Compilation also checks for syntax and type errors. The compiler version should be compatible with the pragma statement.

★ What is the Deploy & Run Transactions panel?

Answer: It is the Remix panel used to select the execution environment, account, gas settings and contract to deploy. It also displays deployed contract instances and allows users to call their functions.

★ What is Remix VM?

Answer: Remix VM is a simulated blockchain environment that runs inside the browser. It provides test accounts and simulated ETH, so contracts can be deployed and tested without connecting MetaMask or spending real funds.

What is the Browser Extension or Injected Provider environment?

Answer: This environment connects Remix to a browser wallet such as MetaMask. The wallet supplies the selected account and network and asks the user to approve transactions. It is useful for deploying to testnets or public networks.

What is the difference between Remix VM and MetaMask deployment?

Answer: Remix VM runs in a local browser simulation and usually executes without wallet pop-ups. MetaMask deployment sends transactions to the selected real or test blockchain network and requires wallet approval and the network's fee currency.

★ Why must MetaMask and Remix use the same network?

Answer: The wallet signs transactions for its currently selected network. If Remix is connected to a different network, the deployment may go to an unintended chain or the expected account and balance may not be available. Always verify the network before deploying.

What does the account field in Remix show?

Answer: It shows the account used by the selected environment to deploy contracts and send transactions. With a browser wallet, the account is generally selected in MetaMask. The account needs sufficient funds for the network's gas fees.

What is the Value field in Remix?

Answer: The Value field specifies how much Ether is sent with a deployment or function call, when the contract allows it. It is not the gas fee. For a non-payable function, the value should normally remain zero.

What is the Deployed Contracts section?

Answer: It lists contract instances deployed or loaded in the current Remix session. Expanding an instance displays callable functions and variables from the contract ABI, allowing the user to test the contract.

What is the difference between Deploy and At Address in Remix?

Answer: Deploy creates a new contract instance through a deployment transaction. At Address loads an already deployed contract at a specified address, using the matching ABI or source interface. It does not create a new contract.

8. Bank Account and Student Data Practicals

★ What is the purpose of the Bank Account smart contract practical?

Answer: The practical demonstrates how a smart contract can maintain account-related data or Ether balances and expose functions to perform operations. It helps understand state variables, functions, access rules and payable transactions.

How can a smart contract accept a deposit?

Answer: A contract can provide a payable deposit function. When a user calls it with Ether, msg.value contains the amount in Wei. The contract can update its stored balance or account record.

★ How is a withdrawal generally implemented in a Bank Account contract?

Answer: A withdrawal function checks that the requested amount is valid and that sufficient funds are available. It then updates the stored balance and transfers Ether to the intended recipient. Secure checks and careful transfer handling are essential.

Why should a withdrawal check the available balance?

Answer: The check prevents a user from withdrawing more than the recorded balance. Without it, the contract's accounting could become incorrect or the transfer could fail. A condition such as require(balance >= amount) is commonly used.

What is the purpose of a balance variable in a Bank Account contract?

Answer: A balance variable stores the amount associated with an account or user. It may be a single uint or a mapping from addresses to balances. Its exact meaning depends on the contract's implementation.

★ What is the purpose of the Student Data smart contract practical?

Answer: It demonstrates storing and retrieving student records using Solidity data structures. A record may include fields such as student ID, name, age and course. It illustrates structs, arrays, functions and events.

★ Why is a struct useful in the Student Data contract?

Answer: A struct groups related student fields into one logical record. Instead of maintaining separate collections for ID, name, age and course, the contract can store them together as one Student value.

Why might a dynamic array be used to store students?

Answer: A dynamic array can hold a changing number of student records. Each new record can be appended using push(), and records can be accessed by index. The array is useful when the contract needs to maintain a list.

What is the purpose of an addStudent() function?

Answer: It accepts student details and stores them in the contract, often by creating a Student struct and adding it to an array. It may also emit a StudentAdded event so external applications can detect the addition.

★ Why should student IDs be checked before adding a record?

Answer: A check can prevent duplicate IDs and maintain record consistency. The contract may search existing records or maintain a mapping to track whether an ID is already used. The exact approach depends on the implementation.

9. Security, Errors and Higher-Level Viva Questions

★ Why should you never share your MetaMask Secret Recovery Phrase?

Answer: The phrase can restore access to the wallet and its accounts. Anyone who obtains it may transfer the wallet's assets or sign transactions. Genuine support teams should never need the phrase.

What is phishing in the context of crypto wallets?

Answer: Phishing is an attempt to trick users into revealing secrets or approving harmful transactions through fake websites, messages or applications. Users should verify URLs, review wallet prompts and never enter recovery phrases into untrusted pages.

What is a reentrancy attack?

Answer: A reentrancy attack occurs when an external call allows another contract to call back into the vulnerable contract before its first operation has safely completed. It can cause repeated withdrawals or incorrect state changes. Checks-effects-interactions and reentrancy guards help reduce the risk.

What is integer overflow and underflow?

Answer: Overflow occurs when an arithmetic result exceeds the maximum value a type can store; underflow occurs when it goes below the minimum. Solidity 0.8 and later checks arithmetic by default and reverts on overflow or underflow unless unchecked arithmetic is used.

Why is access control important in smart contracts?

Answer: Access control restricts sensitive functions to authorized accounts. For example, only an owner or administrator may be allowed to change important settings. Without suitable checks, an attacker may call functions that should be restricted.

What is the Ownable pattern?

Answer: Ownable is a common access-control design in which a contract has an owner address. Functions can use an onlyOwner check to restrict certain operations. It is a design pattern and is often implemented using a library or custom code.

What is an oracle in blockchain?

Answer: An oracle supplies external information to a smart contract, such as prices, weather or real-world events. Smart contracts cannot directly fetch arbitrary off-chain data, so oracles provide a bridge between blockchain and external systems.

What is a 51% attack?

Answer: A 51% attack refers to an attacker controlling a majority of a network's consensus power in a way that may enable certain forms of transaction reordering or censorship. The exact risk depends on the consensus mechanism and network.

Quick reference

- 1 ETH = 10^18 Wei
- 1 Gwei = 10^9 Wei = 10^-9 ETH
- 1 ETH = 10^9 Gwei
- Gas fee depends on gas consumed and the effective fee per gas unit.
- A transaction that changes blockchain state generally requires gas.

Official references for further reading

Ethereum Ether denominations and transfers: https://ethereum.org/developers/docs/intro-to-ether/

Ethereum gas and fees: https://ethereum.org/developers/docs/gas/

Ethereum transactions: https://ethereum.org/developers/docs/transactions/

Solidity documentation: https://docs.solidity.org/

Remix Deploy & Run documentation: https://remix-ide.readthedocs.io/en/latest/run.html